Overview:

For Lab Two, I integrated an AST printer, which accepts a hard-coded AST and prints it in a LISP format. My design parallels the Lox implementation, and currently works in isolation since the parser is not yet implemented. It utilizes the same token classes defined in Lab One, and expands on them by creating links and nodes according to the syntactical grammar design.

Extra Credit Request:
For my language, I also implemented several features that go beyond a basic implementation.

My printer has 3 modes, one to print the standard AST in LISP format, another to print the AST in a tree structure using ASCII characters, and a final mode to print the AST in Reverse Polish Notation. All 3 modes are testable within the attached test file as well.

Syntactical Grammar Design:

My language currently uses the same syntactical grammar design as Lox, as defined below. I chose to retain this grammar because Lab 2 focuses on representing and printing expressions, and the Lox grammar provides the expression types needed by the current implementation.

expression → literal

| unary

| binary

| grouping ;

literal → NUMBER | STRING | "true" | "false" | "nil" ;

grouping → "(" expression ")" ;

unary → ("-" | "!") expression ;

binary → expression operator expression ;

operator → "==" | "!=" | "<" | "<=" | ">" | ">="

| "+" | "-" | "*" | "/" ;

Setup Instructions & Design Explanation:

Since I do not yet have the parser implemented, I left my main src\lu.py file to handle the scanner implementation from Lab 1. The expression classes are defined in src\expr.py, while the core AST printer logic is handled in src\ast_printer.py.

My test files are under test\lab2\ and can therefore be run in the following format from the project root:

python .\test\lab2\test_ast_printer.py

I opted to run the test file directly rather than have it engage with the main lu.py file because the AST printer is being tested independently of the scanner and parser. The parser has not yet been implemented, so the test cases construct the ASTs directly rather than generating them from source code.

Limitations:

All implemented test cases produced the expected output. The AST printer is limited to hard-coded ASTs for now, since the parser isn't yet implemented. The syntactical grammar is currently near-identical to Lox, since I heavily referenced its structure during my implementation. More nuances to syntax will likely emerge as I integrate the parser.

Testing:

For all of the testing, the test cases and hard coded ASTs are provided as Lu input in the file .\test\lab2\test_ast_printer.py There are 7 test cases, each containing a different aspect of the syntactical grammar.

Specific descriptions of the test cases are provided below, and all expected outputs match the actual outputs which are shown below.

TEST 1: Literals

This test case handles basic literal types to ensure that they print properly in the AST printer format.

123

45.67

hello

true

false

nil

TEST 2: Unary

This test case handles basic literal types with a unary operator to ensure that they print properly in the AST printer format.

(- 123)

(! true)

TEST 3: Binary

This test case handles two basic literals with a binary operator to ensure that they print properly in the AST printer LISP format.

(+ 1 2)

(- 5 3)

(* 4 6)

(/ 8 2)

TEST 4: Comparison and Equality

This test case handles two basic literals with a comparison or equality operator to ensure that they print properly in the AST printer LISP format.

(== 1 1)

(!= 1 2)

(> 5 3)

(>= 5 5)

(< 2 4)

(<= 2 2)

TEST 5: Grouping

This test case handles basic literals within a group to ensure that they print properly in the AST printer LISP format.

(group 123)

(group (+ 1 2))

TEST 6: Nested Expression

This test case handles the provided example case from the Lab 2 instructions to test all of the AST Printer features at once.

(* (- 123) (group 45.67))

TEST 7: Complicated Nesting

This test case handles another nested example to ensure that all of the AST printer features are connected properly.

(! (== true (group (+ 3 4))))

Within the test file, there is also a commented out portion to change the printer type from LISP representation to either RPN or a Tree. All actual outputs match the expected outputs, which are as follows:

Tree Structure:

```
TEST 1: Literals
Literal (value=123)
Literal (value=45.67)
Literal (value='hello')
Literal (value='true')
Literal (value='false')
Literal (value='nil')

TEST 2: Unary
Unary (operator='-')
└── right: Literal (value=123)
Unary (operator='!')
└── right: Literal (value='true')

TEST 3: Binary
Binary (operator='+')
├── left: Literal (value=1)
└── right: Literal (value=2)
Binary (operator='-')
├── left: Literal (value=5)
└── right: Literal (value=3)
Binary (operator='*')
├── left: Literal (value=4)
└── right: Literal (value=6)
Binary (operator='/')
├── left: Literal (value=8)
└── right: Literal (value=2)

TEST 4: Comparison and Equality
```

```
Binary (operator='==')
├── left: Literal (value=1)
└── right: Literal (value=1)
Binary (operator='!=')
├── left: Literal (value=1)
└── right: Literal (value=2)
Binary (operator='>')
├── left: Literal (value=5)
└── right: Literal (value=3)
Binary (operator='>=')
├── left: Literal (value=5)
└── right: Literal (value=5)
Binary (operator='<')
├── left: Literal (value=2)
└── right: Literal (value=4)
Binary (operator='<=')
├── left: Literal (value=2)
└── right: Literal (value=2)

TEST 5: Grouping
Grouping
└── expression: Literal (value=123)
Grouping
└── expression: Binary (operator='+')
    ├── left: Literal (value=1)
    └── right: Literal (value=2)

TEST 6: Nested Expression
Binary (operator='*')
├── left: Unary (operator='-')
│   └── right: Literal (value=123)
└── right: Grouping
    └── expression: Literal (value=45.67)

TEST 7: Complicated Nesting
Unary (operator='!')
└── right: Binary (operator='==')
    ├── left: Literal (value='true')
    └── right: Grouping
        └── expression: Binary (operator='+')
            ├── left: Literal (value=3)
            └── right: Literal (value=4)
```

RPN:

TEST 1: Literals
123
45.67
hello
true
false
nil

TEST 2: Unary
123 -
true !

TEST 3: Binary
1 2 +
5 3 -
4 6 *
8 2 /

TEST 4: Comparison and Equality
1 1 ==
1 2 !=
5 3 >
5 5 >=
2 4 <
2 2 <=

TEST 5: Grouping
123
1 2 +

TEST 6: Nested Expression
123 - 45.67 *

TEST 7: Complicated Nesting
true 3 4 + == !